Contents

Presentation Q&A — anticipated questions and answers

A cheat sheet for the Thursday presentation. Organised by topic; each entry has the short answer first, then the math / code reference if you need to drill down. Cross-references to README sections in `[brackets]`.

---

1. The COS method itself

Q1. In one sentence, what is the COS method?

A way to price European options by writing the risk-neutral density as a Fourier-cosine series on a finite truncated interval $[a, b]$, where the series coefficients can be read directly off the model's characteristic function — turning the price into a single inner product.

Q2. What's the headline formula?

$$V(x,t) = K\, e^{-r\tau}\, \mathrm{Re}\left[\sum_{k=0}^{N-1}{}' \varphi\Big(\frac{k\pi}{b-a}\Big)\, e^{ik\pi(x-a)/(b-a)}\, V_k\right]$$

- φ: characteristic function of $\log(S_T/F)$ - V_k: analytic payoff coefficients (chi/psi from Eqs. 22-23 of the paper)
- $[a, b]$: truncation interval, set from the density's cumulants - $\sum'$: prime sum — the $k = 0$ term is halved

Q3. Where does the cosine series come from?

Any function on $[a, b]$ that's smooth enough has a Fourier-cosine series:

$$f(x) \;=\; \sum_{k=0}^{\infty}{}' A_k \cos\Big(k\pi\frac{x-a}{b-a}\Big), \qquad A_k = \frac{2}{b-a}\int_a^b f(x)\cos\Big(k\pi\frac{x-a}{b-a}\Big)\,dx.$$

The key identity (paper Eq. 8): if f is a probability density, the integral defining A_k is the **real part of the characteristic function evaluated at $u_k = k\pi/(b-a)$, multiplied by a phase**:

$$A_k \;\approx\; \frac{2}{b-a}\,\mathrm{Re}\big[\varphi(u_k)\, e^{-iu_k a}\big].$$

So the cosine coefficients of the unknown density are essentially free once you have the CF.

Q4. Why "prime sum"?

Standard cosine-series convention: the $k = 0$ term is the constant (mean) component, but the orthogonality identity that defines the coefficients gives it twice the weight of the higher modes. The prime $\sum'$ notation means "halve the $k = 0$ term" so the reconstruction is correct. In code, we apply the halving once, either to the CF or to V_k (mathematically equivalent).

Q5. What sets the truncation interval $[a, b]$?

Eq. 49 of the paper, from the cumulants of the log-price distribution:

$$b - a = L\sqrt{|c_2| + \sqrt{|c_4|}}, \qquad \text{centered on } c_1.$$

L is a safety multiplier: 10 at $\tau = 1$, growing to 30 at $\tau = 10$ in the paper. We use $L = \max(10, 3\tau + 2)$ to interpolate smoothly. For BSM and VG we have analytic cumulants; for Heston we use the §5.2 simpler heuristic $\sigma \approx \sqrt{\bar{u} + v_0\eta}$ (tighter than the general rule for typical Heston parameter sets).

Q6. What's the convergence rate?

Exponential in N for any density that's analytic on $[a, b]$ — this includes BSM, Heston, CGMY (with appropriate L), and VG at long maturities. Theorem 3.1 of the paper formalises this. At short maturities, VG's CF decays slowly (the density gets a sharp peak) and convergence drops to **algebraic** order ~ 3. This is exactly what we see in Table 7 (T = 0.1 vs T = 1.0).

Q7. Why is COS so much faster than Carr-Madan?

See **README §"Why COS converges faster than the other three"**. Three things compound: 1. **Finite domain**: COS works on $[a, b]$ (a bounded interval), not the whole real line. The cosine coefficients of an analytic density on a bounded interval decay *exponentially* in k. 2. **Analytic payoff coefficients**: the χ and ψ integrals are closed form (Eqs. 22-23). No numerical integration of the payoff. 3. **No damping parameter**: Carr-Madan needs $\alpha > 0$ to make the call payoff Fourier-integrable; this is a tuning tradeoff. COS sidesteps it by integrating the payoff analytically over the bounded interval.

Q8. What are χ_k and ψ_k?

Closed-form integrals from paper Eqs. 22-23:

$$\chi_k(c, d) = \int_c^d e^x \cos\Big(k\pi\frac{x-a}{b-a}\Big)\, dx, \qquad \psi_k(c, d) = \int_c^d \cos\Big(k\pi\frac{x-a}{b-a}\Big)\, dx.$$

Both have closed forms (you integrate by parts twice for χ). The vanilla call payoff coefficient is $V_k = (2/(b - a))[\chi_k(0, b) - \psi_k(0, b)]$. The cash-or-nothing digital uses just ψ_k. The Bermudan exercise piece uses both, integrated over the exercise sub-interval.

Q9. Why does the cash-or-nothing payoff (Table 3) still converge exponentially despite being discontinuous?

Standard quadrature methods (and naive FFT) suffer Gibbs oscillations at the discontinuity, giving algebraic convergence. The COS method **integrates the discontinuous payoff exactly** in closed form (just ψ_k, since the payoff is a step), so the only error is in the density approximation — which is still smooth and has exponentially-decaying cosine coefficients. So we get exponential convergence on a discontinuous payoff. This is Theorem 3.1.

---

2. Carr-Madan, Lewis, FrFT (the comparison context)

Q10. What does Carr-Madan actually compute?

The Fourier inversion of the *damped* call price. Define $c_T(k) = e^{\alpha k}C(K = e^k)$ for some damping $\alpha > 0$. Then $\hat{c}_T(v) = \mathcal{F}[c_T](v)$ is well-defined (without damping it isn't — the call value diverges as $k \to -\infty$). Compute $\hat{c}_T(v)$ from the model's CF, then invert via FFT and undamp.

Q11. Why does Carr-Madan need a damping parameter?

Because the call payoff $\max(S_T - K, 0)$ is not Fourier-integrable in the log-strike $k = \log K$ — it grows linearly as $k \to -\infty$. Multiplying by $e^{\alpha k}$ ($\alpha > 0$) damps that growth. The price is $C(K) = e^{-\alpha k}/\pi \cdot \int_0^\infty \mathrm{Re}[e^{-ivk}\psi(v)]dv$ where $\psi(v)$ involves the CF at the complex argument $v - i(\alpha + 1)$.

Q12. Why is picking α a tradeoff?

- α too **large**: the integrand $\psi(v)$ involves the CF at a deep complex shift, which can blow up for some models (CGMY, VG with extreme parameters). Also amplifies high-frequency error.
- α too **small**: the damping factor $e^{\alpha k}$ doesn't kick in fast enough; the integrand decays slowly and the truncation error grows.

There's no closed-form optimal α. Carr & Madan recommend $\alpha = 0.75$ as a heuristic.

Q13. What is Lewis (2001) and how does it relate?

Lewis fixes α at $1/2$ — the **symmetric** choice that makes the integrand $\mathrm{Re}[e^{iuk}\phi(u - i/2)]/(u^2 + 1/4)$ have the cleanest analytic structure. At $\alpha = 1/2$ you get a single integral with no parameter to tune, and the $1/(u^2+1/4)$ denominator gives natural decay. Convergence on Gauss-Legendre nodes is **geometric** because the integrand is analytic in u.

Q14. Why doesn't Lewis dominate Carr-Madan in practice if $\alpha = 1/2$ is optimal?

In our Table 2 it actually does — Lewis hits 4e-10 by $N = 256$ while CM is stuck at 6e-4. The catch: Lewis is **per-strike** (no FFT batching across strikes). At M strikes, Lewis costs $M \cdot N$ CF evaluations, vs N for Carr-Madan/FrFT. That's why Lewis's wall time blows up at large M — see Table 2 ms column.

Q15. What does FrFT add over plain Carr-Madan?

Plain FFT enforces $\eta\lambda = 2\pi/N$ (frequency spacing $\times$ log-strike spacing fixed by N). Pick η small (good frequency resolution): λ becomes large (coarse strike grid). Pick η large: λ becomes small but you cap the frequency range.

The **Bailey-Swarztrauber fractional FFT** computes $y_k = \sum_j x_j e^{-2\pi i \beta jk}$ for any real β (not just $\beta = 1/N$). Plug $\beta = \eta\lambda/(2\pi)$ — now η and λ are independent. Pin λ small (fine strike grid) and pick η to cover the integrand support.

Q16. How does FrFT actually do this in three FFTs?

Identity: $jk = (j^2 + k^2 - (j-k)^2)/2$, so $e^{-2\pi i\beta jk} = e^{-i\pi\beta j^2}\, e^{-i\pi\beta k^2}\, e^{i\pi\beta(j-k)^2}$. The sum factorises into a convolution of two chirped sequences:

$$y_k = e^{-i\pi\beta k^2} \cdot \mathrm{conv}(\{x_j e^{-i\pi\beta j^2}\},\ \{e^{i\pi\beta j^2}\})_k.$$

Compute the convolution via standard FFT of length $2N$ (zero-padded). See src/cos_pricing/frft.py `_frft()`.

---

3. Heston-specific

Q17. What does the Heston CF look like?

$$\varphi(u;\tau) = \exp(iu(r-q)\tau + C(u;\tau)\bar{u} + D(u;\tau)v_0)$$

with C, D explicit functions of u, τ, and the model parameters κ (= `lam`), η, ρ. The closed form involves a complex square root $d(u) = \sqrt{(\rho\eta iu - \kappa)^2 + \eta^2 u(u+i)}$ and a complex logarithm.

Q18. What is the "Little Heston Trap"?

Two algebraically equivalent forms of the CF (the original Heston 1993 form and the "trap-free" Albrecher et al. 2007 form) become numerically unstable in different parameter regions because the complex logarithm has a branch cut. The trap-free form (which we use) stays continuous as τ grows. Key reference: Albrecher, Mayer, Schoutens, Tistaert (2007).

Q19. Why use `expm1` and `log1p`?

At long maturities ($\tau = 10$ in Table 5), $e^{-d\tau}$ can be very small, so $1 - e^{-d\tau}$ loses precision in floating-point. `np.expm1(-x)` computes $e^{-x} - 1$ accurately for small x. Similarly for $\log(1 - y)$. The mathematical identities are exact; the benefit is purely floating-point preservation of the last few digits.

Q20. Why do we beat the paper's runtime by an order of magnitude on Heston?

Two big things, plus 5 smaller ones. **Numba** compiles the entire pricing loop (cumulant + CF + payoff + dot product) into one machine-code loop, eliminating Python's per-NumPy-op overhead — a plain NumPy version runs ~30 separate array operations, each paying that overhead. **Result caching** in `HestonCOSPricer` means repeated calls (calibration loops, finite-difference Greeks) return from a dict lookup. The other 5 items are accuracy improvements; see README §"Numerical changes that beat the paper on Heston".

Q21. Why scale L with τ?

Long-maturity Heston densities have fatter tails (variance grows with τ). A fixed L that works for $\tau = 1$ undershoots at $\tau = 10$ — the truncation error swamps the series-truncation error. The paper does this discretely ($L = 10$ at $\tau = 1$, $L = 30$ at $\tau = 10$). We interpolate linearly: $L = \max(10, 3\tau + 2)$.

Q22. Why center $[a, b]$ on $x + c_1$ instead of x?

The Heston density of $\log(S_T/K)$ given $\log(S_0/K) = x$ has mean $x + c_1$. Centering $[a, b]$ on the actual mean equalises the probability mass captured in each tail; centering on x leaves more mass in one tail than the other, increasing the truncation error at fixed L. This is the standard centering used by Ruijter & Oosterlee (2012).

---

4. VG and CGMY

Q23. What's the VG CF?

$$\varphi(u; T) = \exp(iuwT)\,(1 - iu\theta\nu + 0.5\,\sigma^2 u^2 \nu)^{-T/\nu},$$

where $w = \log(1 - \theta\nu - 0.5\sigma^2\nu)/\nu$ is the martingale drift correction. ν has units of time, θ has units of 1/time, σ^2 has units of 1/time.

Q24. Why is VG algebraic at short T but exponential at long T?

At short T, the CF decays only as $|u|^{-2T/\nu}$ — for $T = 0.1$, $\nu = 0.2$, that's $|u|^{-1}$. The cosine coefficients of the density inherit this slow decay, so the COS series error is **algebraic** in N (order ~ 3). At long T (e.g. $T = 1.0$, $\nu = 0.2$), the exponent becomes $|u|^{-10}$, decay is fast, density is smooth, error is **exponential** in N.

Q25. What's the CGMY CF?

$$\varphi(u; T) = \exp\Big(iuwT + T\,C\,\Gamma(-Y)\,[(M - iu)^Y - M^Y + (G + iu)^Y - G^Y]\Big)$$

where $w = -C\Gamma(-Y)\,[(M - 1)^Y - M^Y + (G + 1)^Y - G^Y]$ is the martingale drift correction. C controls overall jump intensity (units: 1/time), G and M control the rate of jump-size decay (left and right tail), Y controls how fine the jumps are ($Y < 0$: finite activity; $0 < Y < 1$: infinite activity, finite variation; $1 < Y < 2$: infinite activity AND infinite variation).

Q26. Why does Y = 1.98 break the paper's published bounds (Table 10)?

For Y very close to 2, the martingale drift w becomes very large in magnitude (we measured $w \approx -87.5$). The density gets shifted heavily negative, and with the paper's stated bound $[-100, 20]$ the density is squeezed against the left edge. We can match the paper at $N = 25$ but plateau at $\sim 8 \times 10^{-3}$ from $N = 30$ onward because the boundary truncates the left tail. The paper reports machine precision at $N = 40$ — almost certainly using a wider unpublished bound (e.g. $[-300, 20]$).

Q27. Why doesn't VG / CGMY get analytic cumulants for free?

They do, in the paper (VG: Table 11, CGMY: Eq. 56). We use them for the truncation range. The reason setting $[a, b]$ from cumulants is preferred over numerical-derivative estimation: the analytic formulas are exact and stable; numerical differentiation of $\log \varphi$ accumulates error.

---

5. Bermudan COS (Fang-Oosterlee 2009)

Q28. What's the algorithm in 4 steps?

1. **Initialise** at terminal time $t_M = T$ with European payoff coefficients (χ / ψ).
2. **Backward induction** from t_{M-1} down to t_1:
   - **Continuation value** at any x: $\hat{V}(x, t_j) = e^{-r\Delta t}\,\mathrm{Re}[\sum_n{}' V_n(t_{j+1})\varphi_{\Delta t}(u_n)e^{iu_n(x-a)}]$ — same formula as European pricing, just with the next-step coefficients as the "payoff."
   - **Find boundary** x^*: solve $\hat{V}(x^*) = \text{intrinsic}(x^*)$ via Brent's method. For a put, exercise on $[a, x^*]$, continue on $[x^*, b]$.
   - **New coefficients** $V_k(t_j) = G_k(x^*) + C_k(x^*)$:
     - G_k — exercise piece: analytic via χ / ψ on the exercise interval.
     - C_k — continuation piece: closed-form $\mathcal{M}_{k,n}(x^*)$ matrix product.
3. **Final step**: option value at \$t_0 = \$ continuation value at $x_0 = \log(S_0/K)$ (no exercise at inception).

Q29. What's the $\mathcal{M}_{k,n}(x^*)$ matrix?

$$\mathcal{M}_{k,n}(x^*) = \frac{2}{b-a}\int_{x^*}^{b} \cos\left(\frac{k\pi(x-a)}{b-a}\right) e^{in\pi(x-a)/(b-a)}dx.$$

Using $2\cos(\alpha)e^{i\beta} = e^{i(\beta+\alpha)} + e^{i(\beta-\alpha)}$ this splits into two integrals of $e^{i\omega z}$ over a finite interval — closed form. The matrix is complex, $N \times N$, computed once per timestep. See src/cos_pricing/bermudan.py `_M_matrix()`.

Q30. Why does $M = 1$ have to equal the European put exactly?

With one exercise opportunity at T, the Bermudan payoff is identical to the European payoff. Our backward-induction loop runs zero iterations (loop range is empty for $M = 1$); the single "final step" computes the continuation value, which is by construction the European COS price. So $M = 1$ Bermudan = European. We assert this to **machine epsilon (1e-14)** as a structural validation — catches sign errors, prime-sum bugs, CF-convention mistakes anywhere in the backward-induction path.

Q31. Why is the price monotonically non-decreasing in M?

More exercise opportunities cannot reduce option value (you can always *not* exercise and get back the previous Bermudan). Asserted in tests as a sanity check. If the curve dips, something is wrong (most likely numerical noise from too-small N for the requested M).

Q32. What's the published American put for our test case?

$S = K = 100$, $r = 0.1$, $q = 0$, $\sigma = 0.25$, $T = 1$: standard binomial-tree American put converges to ~ 6.55. Our $M = 200$ Bermudan is at 6.5508 — extrapolating the geometric decay of the increments, the limit is in $[6.555, 6.560]$. Matches the literature.

Q33. Cost of the algorithm?

$O(MN^2)$ per option (the $\mathcal{M}_{k,n}$ matrix is $N \times N$ per timestep). At $N = 128$, $M = 64$ that's about a million ops per option — runs in ~50 ms per call. The 2009 paper notes this can be reduced to $O(MN \log N)$ via the Hankel-plus-Toeplitz structure of $\mathcal{M}$ + FFT — we don't bother because $O(MN^2)$ is already sub-second at typical scale.

Q34. Does it handle calls?

Yes — recently added. Call early exercise only matters with $q > 0$ (otherwise the no-early-exercise theorem says American call = European call). The algorithm is symmetric: exercise on $[x^*, b]$ instead of $[a, x^*]$, continue on the complementary interval. See `BermudanCosBSM.price_call()`.

---

6. Dimensional analysis (Buckingham π)

Q35. What is Buckingham's π theorem?

A physical / mathematical result: if a system has n inputs spanning r independent base units, then it can be fully described by $n - r$ dimensionless groups (π-groups). For BSM: 5 inputs (S_0, K, r, σ, T), 2 base units ($ and time), so 3 π-groups: K/S_0, $\sigma\sqrt{T}$, rT.

Q36. Why does scale invariance hold by construction?

The kernel of every exponential-Lévy pricer in this repo evaluates the CF at frequencies $u_k = k\pi/(b-a)$, where $[a, b]$ is in **log-moneyness** coordinates $x = \log(S/K)$. Scaling spot and strike by the same factor λ leaves $\log(\lambda S/\lambda K) = \log(S/K)$ unchanged — the kernel sees bit-identical numbers. So $C(\lambda S, \lambda K) = \lambda C(S, K)$ holds to machine precision, not as an approximation.

Q37. What's the temporal symmetry for Heston?

$$C(T, r, q, \kappa, \eta, v_0, \bar{v}) \;=\; C(\mu T, r/\mu, q/\mu, \kappa/\mu, \eta/\mu, v_0/\mu, \bar{v}/\mu).$$

Seven parameters all rotate at once. The change of variable $\tau' = t/\mu$ in the Heston SDEs combined with Brownian rescaling returns the original SDE in τ'-time, so the law of $\log(S_T/S_0)$ is the same under both parameterisations.

Q38. Why is CGMY temporal invariance bit-identical (zero error) but VG isn't?

CGMY's rescaling only changes C (linearly) and the rates — no irrational-number scaling. VG's $\sigma \to \sigma/\sqrt{\mu}$ involves a square root, which breaks bit-identity at the floating-point level. Both are below 1e-15 (well within machine epsilon) — the CGMY happens to land *exactly* on zero.

Q39. Why is dimensional analysis useful in practice?

Three concrete uses (README §"Why this matters"): 1. **Correctness tests without analytic references** — if a pricer breaks the predicted symmetry, something's wrong in the kernel. 2. **Calibration in dimensionless coordinates** — the price surface is a function of π-groups only, so one calibration covers infinitely many raw-parameter combinations. 3. **Sanity check across maturities** — temporal invariance pins down equivalent calibrations at different T.

Q40. Why does the README emphasise BSM/Heston/VG/CGMY only?

Because those are the four models the Fang-Oosterlee paper covers. We previously had Bachelier as a 5th example, but Bachelier is *additive* (translation-invariant), not multiplicative — its π-symmetry is structurally different and off-scope vs. the paper. Per Erce's professor's feedback, we dropped it and added VG/CGMY in the dim-analysis story to match the paper.

---

7. Implementation details

Q41. Why numba? What does it actually do?

The Heston pricing kernel is a tight loop over $k = 0, \ldots, N-1$ doing complex arithmetic. In plain NumPy, each line of the loop allocates a temporary array and pays Python interpreter overhead — for $N = 200$ that's ~30 array operations per call. Numba JIT-compiles the loop ahead of time into native machine code (LLVM), so the loop runs at C-like speed. Results: ~10 µs per Heston option vs ~150 µs for a NumPy version.

Q42. Why cache the price array, not just intermediate computations?

Because in calibration loops and finite-difference Greeks, the *exact same* $(K, \tau, N, L, \text{cp})$ tuple gets requested multiple times. A simple dict lookup (~1 µs) replaces the entire kernel call. We use FIFO eviction at 64 entries to bound memory under varied calibration sweeps.

Q43. How does `cos_method.cos_price` handle puts?

Two paths: (1) compute call coefficients via χ / ψ, then derive put coefficients via the analytic put-call parity correction — single $O(N)$ adjustment; (2) for scalar `cp`, fast-path through the appropriate code branch directly. Both produce the same answer.

Q44. Why the prime-sum convention "halve V_0" instead of "halve the CF"?

Mathematically equivalent: the COS sum is $\sum_k' a_k b_k$ where a_k and b_k are real or complex; halving a_0 or b_0 gives the same result. We halve the CF in cos_method.py (line 86: `cf_s[0] *= 0.5`); we halve V_0 in the Bermudan recurrence because the Bermudan recurrence reuses $V_n(t_{j+1})$ many times and halving the CF would require re-applying it each step.

Q45. Why is the test suite 174 tests?

112 (original baseline) → 142 (added 30 dim-analysis tests for VG/CGMY) → 149 (added 7 Lewis tests) → 164 (added 6 FrFT + 9 Bermudan) → 174 (added 10 call-side Bermudan tests). The structural validations (M=1 = European, monotonicity, geometric convergence, π-symmetry) are the most informative — they fail if anything fundamental is wrong, even without an external benchmark.

Q46. Why include PyFENG integration?

PyFENG is Prof. Jaehyuk Choi's research package. Our `pyfeng/sv_cos.py` follows PyFENG's `CosABC` / `BsmCos` / `HestonCos` class hierarchy exactly, so it's a drop-in alongside `HestonFft`. This means our COS implementation can be benchmarked directly against PyFENG's existing FFT pricer, and downstream PyFENG users can swap pricers transparently.

---

8. Likely "gotcha" questions

Q47. "Your COS method beats the paper at small N — are you cherry-picking?"

We use **analytic BSM cumulants** ($c_1 = -\sigma^2 T/2$, $c_2 = \sigma^2 T$, $c_4 = 0$) for the truncation range, giving the tightest possible $[a, b]$. The paper uses a wider, more conservative range (specifically as a stress test in some tables). Both implementations confirm the same exponential-convergence behaviour predicted by Theorem 3.1; we just hit machine precision earlier because the integration window is tighter.

Q48. "FrFT looks barely better than Carr-Madan in your Table 2. Why include it?"

At small N the dominant error is the frequency truncation — strike-grid resolution doesn't matter yet. At $N = 512$, FrFT is ~7× tighter than plain CM because the strike grid stays fine. More importantly, FrFT is **strictly stronger** than CM (it reduces to CM exactly when $\beta = 1/N$) — including it shows the comparison is fair and exhaustive.

Q49. "Lewis has $N = 128$ at 3.5e-6 error — how can it be 'better' than CM at the same N?"

Different convergence regimes. At $N = 128$: Lewis = 3.5e-6, CM = 7.8e-2. **Geometric vs algebraic.** Each doubling of N buys Lewis ~3-6 orders of magnitude vs ~1 order for CM. By $N = 256$ Lewis is at 4e-10; CM is at 6e-4.

Q50. "Why didn't you implement Bermudan for Heston?"

Heston log-returns aren't i.i.d. (they depend on the variance state v_t), so the single-state COS recurrence doesn't apply directly. The Heston Bermudan extension is **2-D COS** (Ruijter & Oosterlee 2012), where you track both log-spot and variance jointly. That's substantially more code (the matrix $\mathcal{M}_{k,n}$ becomes $\mathcal{M}_{k_1,k_2,n_1,n_2}$ — N^4 entries) and out of scope for this project. We mention this in the README as a natural extension.

Q51. "Your dim-analysis errors are all under 1e-15. Aren't those just floating-point noise?"

Yes — and that's the **point**. The π-symmetries hold *by construction* in the kernel, so the only error is single-ulp floating-point drift from operations that aren't bit-symmetric (e.g. computing $F = Se^{(r-q)T}$ vs $\lambda F = (\lambda S)e^{(r-q)T}$ involves separate `exp()` calls). When $r = q$, even those drifts vanish (we get exact zeros). The non-zero cells in our spatial-scale table are 1-ulp differences in the carry factor.

Q52. "Carr-Madan needs $\alpha = 0.75$ — where does that number come from?"

It's a heuristic from the original Carr-Madan paper (Remark 5.1), based on a balance between damping enough to make the integrand integrable but not so much that the CF at $v - i(\alpha + 1)$ has numerical issues. There's no closed-form optimum. Some authors use $\alpha = 1.5$; QuantLib defaults to $\alpha = 1.25$. We use 0.75 to match the paper's setup so the comparison is apples-to-apples.

Q53. "Why does Heston Section 5 use 'lam' for κ?"

Naming convention from the original Fang-Oosterlee paper. The paper uses λ for mean-reversion speed (not eigenvalues). We follow the paper's notation in the API to make the cross-reference unambiguous. PyFENG uses `mr` (for "mean reversion") instead, which is what `pyfeng/sv_cos.py` adopts.

Q54. "The paper claims the truncation rule is for general densities — does it actually work for the fat-tailed CGMY?"

Mostly. For $Y \in [0, 1.5]$, the cumulant-based rule with $L = 10$ works fine. For Y very close to 2 (Table 10), the strict martingale shift moves the conditional mean far negative and the cumulant-based rule undersizes the left tail. The paper's prescription $[-100, 20]$ doesn't quite work for our implementation — we believe the paper's authors used a wider unpublished bound. For practical use, $Y \in [0.5, 1.7]$ is the well-behaved range.